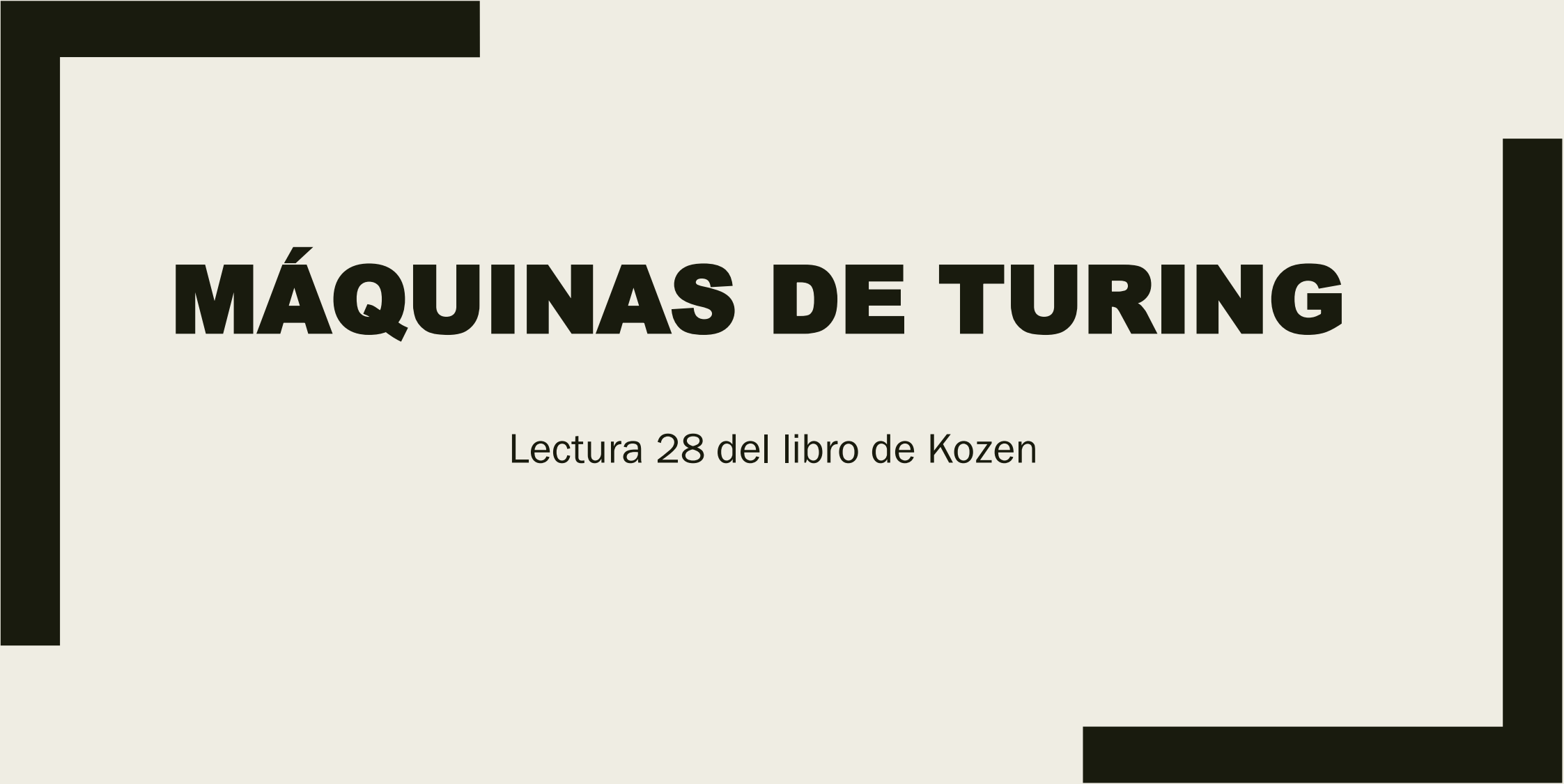


LENGUAJES FORMALES DECIDIBILIDAD Y SEMIDECIDIBILIDAD MÁQUINAS DE TURING

Ing. Adolfo Andrés Castro Sánchez, MSc
2026 - I

Área Ciencias Computacionales
Escuela de Ciencias Aplicadas e Ingeniería

UNIVERSIDAD
EAFIT

A thick black L-shaped frame is positioned on the left and bottom edges of the slide, framing the central text.

MÁQUINAS DE TURING

Lectura 28 del libro de Kozen

Introducción

- Máquinas de Turing (TMs), inventadas por Alan Turing en 1936. El autómatas que reconoce lenguajes recursivamente enumerables, recursivos y co-recursivos. Pueden resolver cualquier función calculable.
- **Cálculo efectivo:** Métodos para demostrar lo computacionalmente calculable.
 - *Las máquinas de Turing (Manipulan cadenas sobre un alfabeto finito)*
 - *Máquina de post (La lógica utiliza la teoría de colas)*
 - *Recursividad (Trabajan sobre números naturales),*
 - *Cálculo lambda (Función, aplicación y recursividad)*
 - *Lógica combinacional*
- Variaciones del modelo Turing: No determinístico, Multicinta, cinta multidimensional, bidireccional con cinta infinita

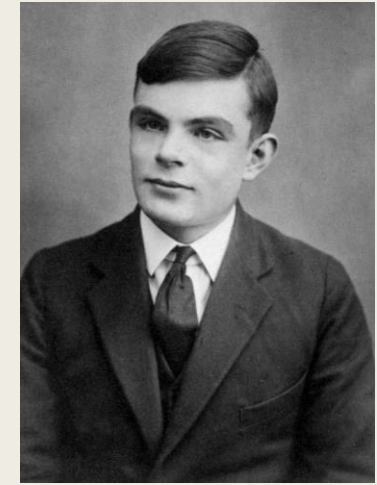
Introducción

However, there are natural translations between all these different types of data. For example, there is a simple one-to-one correspondence between strings in $\{0, 1\}^*$ and natural numbers $\mathbb{N} = \{0, 1, 2, \dots\}$ defined by

$$x \mapsto \#(1x) - 1,$$

where $\#y$ is the natural number represented by the binary string y . Conversely, it is easy to encode just about anything (natural numbers, λ -terms, strings in $\{0, 1, 2, \dots, 9\}^*$, trees, graphs, ...) as strings in $\{0, 1\}^*$. Under these natural encodings of the data, it turns out that all the formalisms above can simulate one another, so despite their superficial differences they are all computationally equivalent.

Tesis Church's-Turing



- Cualquier función que se pueda calcular mediante un proceso mecánico puede ser computada por una máquina de Turing.
- Cálculo lambda: descripción de funciones computables.
- Máquina de Turing: un modelo abstracto de computación de un proceso algorítmico.
- Las ciencias computacionales se derivan de las matemáticas discretas
- Generar una universalidad del estudio computacional
- Modelos que permitan ser implementados
- Equivalencia computacional
- Todo algoritmo es equivalente a una máquina de Turing

Universalidad

Tiene las siguientes características

- Programas como datos
 - *Programas que entienden y manipulan otros*
 - *Codificación de datos en distintas formas*
 - Calculo lambda
 - Lógica combinacional
 - Funciones recursivas
 - Interpretan las cadenas de entrada
- Concepto de la noción de la “Simulación Universal”

versal program or machine U is constructed to take an encoded description of another program or machine M and a string x as input and perform a step-by-step simulation of M on input x . A modern-day example of this phenomenon would be a SCHEME interpreter written in SCHEME.

Autoreferencia

Se define como la capacidad que llevó al descubrimiento de problemas naturales incalculables:

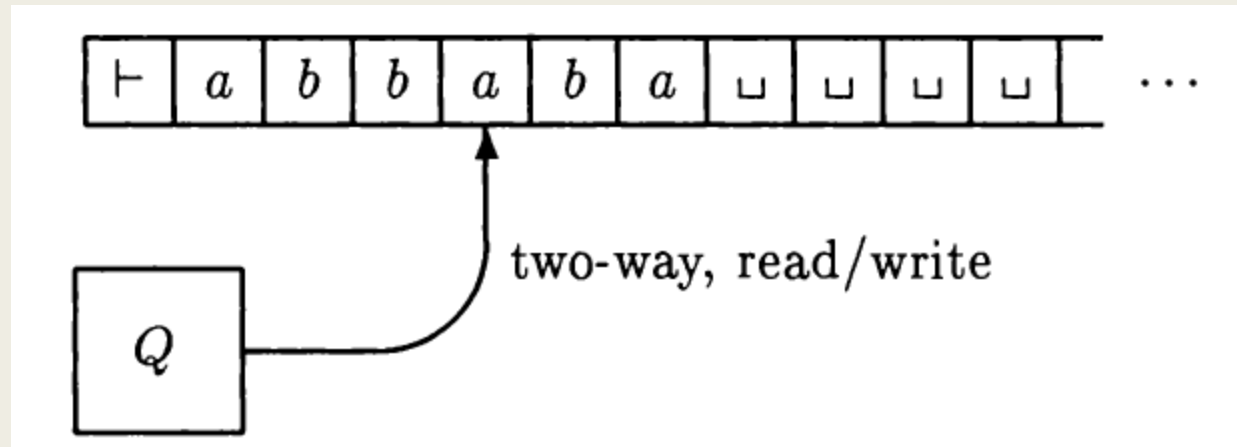
- Permite construir de manera muy simple los ejemplos de problemas incalculables.
- Escenarios que un lenguaje de programación o compilador no podría resolver.
- Limitaciones intrínsecas de los sistemas. Ejemplo: resultados inesperados o paradójicos.

La computación no es completa ni perfecta

- Clasificación de Chomsky

Tipo	Lenguaje	Autómata	Reglas gramaticales
0	LRE	Máquina de Turing	$\alpha \rightarrow \beta$ (sin restricciones)
1	LSC	A. Linealmente Acotado	$\alpha A \beta \rightarrow \alpha \gamma \beta$
2	LLC	A. Con pila	$A \rightarrow \gamma$
3	LR	A. finito	$A \rightarrow aB$ o $A \rightarrow a$

Definición Informal



Definición Formal

Formally, a *deterministic one-tape Turing machine* is a 9-tuple

$$M = (Q, \Sigma, \Gamma, \vdash, \sqcup, \delta, s, t, r),$$

- Q is a finite set (the *states*);
- Σ is a finite set (the *input alphabet*);

- Γ is a finite set (the *tape alphabet*) containing Σ as a subset;
- $\sqcup \in \Gamma - \Sigma$, the *blank symbol*;
- $\vdash \in \Gamma - \Sigma$, the *left endmarker*;
- $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$, the *transition function*;
- $s \in Q$, the *start state*;
- $t \in Q$, the *accept state*; and
- $r \in Q$, the *reject state*, $r \neq t$.

Intuitively, $\delta(p, a) = (q, b, d)$ means, “When in state p scanning symbol a , write b on that tape cell, move the head in direction d , and enter state q .” The symbols L and R stand for left and right, respectively.

We restrict TMs so that the left endmarker is never overwritten with another symbol and the machine never moves off the tape to the left of the endmarker; that is, we require that for all $p \in Q$ there exists $q \in Q$ such that

$$\delta(p, \vdash) = (q, \vdash, R). \tag{28.2}$$

We also require that once the machine enters its accept state, it never leaves it, and similarly for its reject state; that is, for all $b \in \Gamma$ there exist $c, c' \in \Gamma$ and $d, d' \in \{L, R\}$ such that

$$\begin{aligned} \delta(t, b) &= (t, c, d), \\ \delta(r, b) &= (r, c', d'). \end{aligned} \tag{28.3}$$

We sometimes refer to the state set and transition function collectively as the *finite control*.

Ejemplo: Realizar una TM para el conjunto de cadenas:

$$\{a^n b^n c^n \mid n \geq 0\}$$

$$Q = \{s, q_1, \dots, q_{10}, t, r\},$$

$$\Sigma = \{a, b, c\},$$

$$\Gamma = \Sigma \cup \{\vdash, \sqcup, \neg\}.$$

There is nothing special about $\neg$; it is just an extra useful symbol in the tape alphabet. The transition function δ is specified by the following table:

	$\vdash$	a	b	c	$\sqcup$	$\dashv$
s	$(s, \vdash, R)$	(s, a, R)	(q_1, b, R)	(q_2, c, R)	$(q_3, \dashv, L)$	$-$
q_1	$-$	$(r, -, -)$	(q_1, b, R)	(q_2, c, R)	$(q_3, \dashv, L)$	$-$
q_2	$-$	$(r, -, -)$	$(r, -, -)$	(q_2, c, R)	$(q_3, \dashv, L)$	$-$
q_3	$(t, -, -)$	$(r, -, -)$	$(r, -, -)$	$(q_4, \sqcup, L)$	$(q_3, \sqcup, L)$	$-$
q_4	$(r, -, -)$	$(r, -, -)$	$(q_5, \sqcup, L)$	(q_4, c, L)	$(q_4, \sqcup, L)$	$-$
q_5	$(r, -, -)$	$(q_6, \sqcup, L)$	(q_5, b, L)	$-$	$(q_5, \sqcup, L)$	$-$
q_6	$(q_7, \vdash, R)$	(q_6, a, L)	$-$	$-$	$(q_6, \sqcup, L)$	$-$
q_7	$-$	$(q_8, \sqcup, R)$	$(r, -, -)$	$(r, -, -)$	$(q_7, \sqcup, R)$	$(t, -, -)$
q_8	$-$	(q_8, a, R)	$(q_9, \sqcup, R)$	$(r, -, -)$	$(q_8, \sqcup, R)$	$(r, -, -)$
q_9	$-$	$-$	(q_9, b, R)	$(q_{10}, \sqcup, R)$	$(q_9, \sqcup, R)$	$(r, -, -)$
q_{10}	$-$	$-$	$-$	(q_{10}, c, R)	$(q_{10}, \sqcup, R)$	$(q_3, \dashv, L)$

Configuración de aceptación

The *start configuration* on input $x \in \Sigma^*$ is the configuration
 $(s, \vdash x \sqcup^\omega, 0)$.

One can define a *next configuration relation* $\xrightarrow[M]{1}$ as with PDAs. For a string $z \in \Gamma^\omega$, let z_n be the n th symbol of z (the leftmost symbol is z_0), and let $s_b^n(z)$ denote the string obtained from z by substituting b for z_n at position n . For example,

$$s_b^4(\vdash b a a a c a b c a \cdots) = \vdash b a a b c a b c a \cdots.$$

The relation $\xrightarrow[M]{1}$ is defined by

$$(p, z, n) \xrightarrow[M]{1} \begin{cases} (q, s_b^n(z), n-1) & \text{if } \delta(p, z_n) = (q, b, L), \\ (q, s_b^n(z), n+1) & \text{if } \delta(p, z_n) = (q, b, R). \end{cases}$$

We define the reflexive transitive closure $\xrightarrow{*}_M$ of $\xrightarrow{1}_M$ inductively, as usual:

- $\alpha \xrightarrow{0}_M \alpha$,
- $\alpha \xrightarrow{n+1}_M \beta$ if $\alpha \xrightarrow{n}_M \gamma \xrightarrow{1}_M \beta$ for some γ , and
- $\alpha \xrightarrow{*}_M \beta$ if $\alpha \xrightarrow{n}_M \beta$ for some $n \geq 0$.

The machine M is said to *accept* input $x \in \Sigma^*$ if

$$(s, \vdash x \sqcup^\omega, 0) \xrightarrow{*}_M (t, y, n)$$

for some y and n , and *reject* x if

$$(s, \vdash x \sqcup^\omega, 0) \xrightarrow{*}_M (r, y, n)$$

Diferencias entre Máquina de Turing total y Máquina de Turing

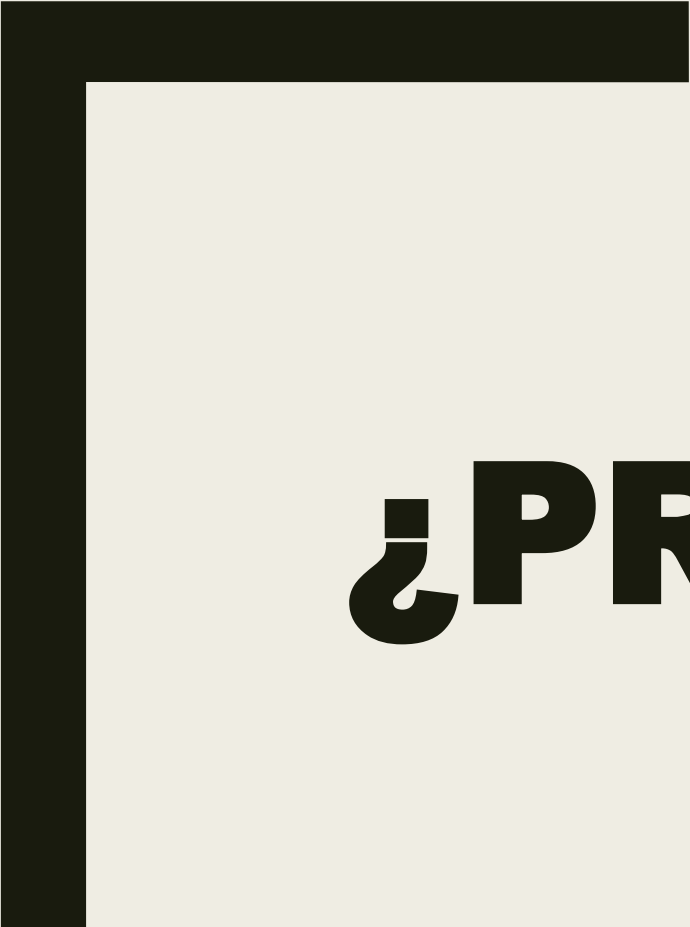
for some y and n . It is said to *halt* on input x if it either accepts x or rejects x . As with PDAs, this is just a mathematical definition; the machine doesn't really grind to a halt in the literal sense. It is possible that it neither accepts nor rejects, in which case it is said to *loop* on input x . A Turing machine is said to be *total* if it halts on all inputs; that is, if for all inputs it either accepts or rejects. The set $L(M)$ denotes the set of strings accepted by M .

Conjuntos de Cadenas

- *recursively enumerable* (r.e.) if it is $L(M)$ for some Turing machine M ,
 - *co-r.e.* if its complement is r.e., and
-
- *recursive* if it is $L(M)$ for some *total* Turing machine M .

Actividad en clase

- Procesar la cadena: *aaabbbccc* por medio de la configuración de aceptación
- Realizar la lectura 29 del libro de Kozen y tomar los apuntes más importantes de los ejemplos planteados.



¿PREGUNTAS?

